UWE Bristol | University of the West of England

Lecture 2: Software development methodologies

Week 2

Last week

- Module Introduction
- Overview of software methodologies

Outline

- Recap Lecture 1
- Software Development Methodologies in detail- Agile approach (Scrum), DevOps

Recap- Lecture 1

Software development methodologies

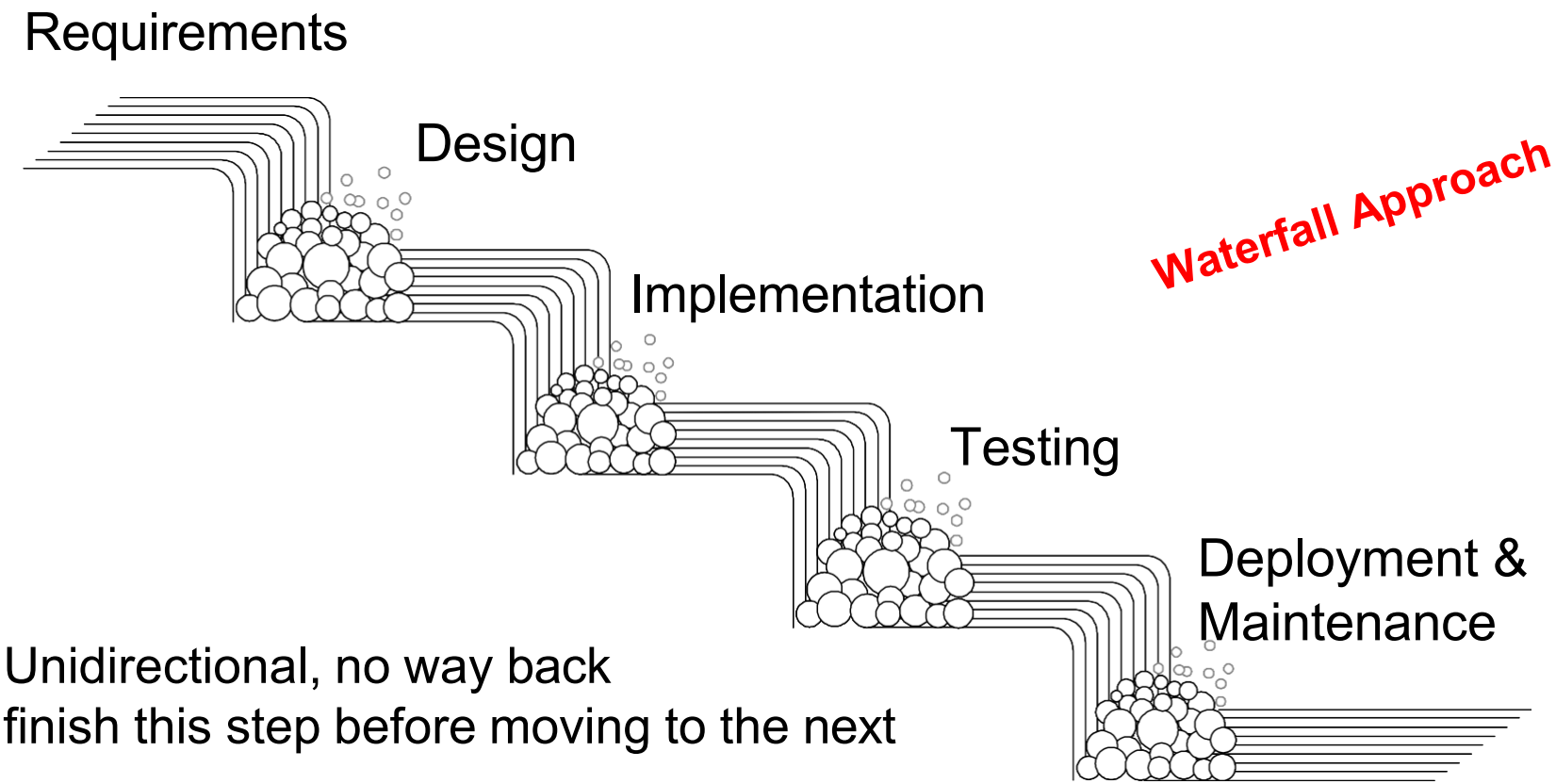
UWE Bristol University of the West of England

- is a framework that can be used to plan, structure and control the process of developing a software system.
- it may have several phases which may run in sequence or in cyclic/iterative way
- Different roles/actors may contribute at different phases
- Several deliverables or artifacts (e.g., documentation, software) may be produced at different phases and how to manage/maintain them
- Guidelines for communication between team or stakeholders

Examples of Software Development Methodologies

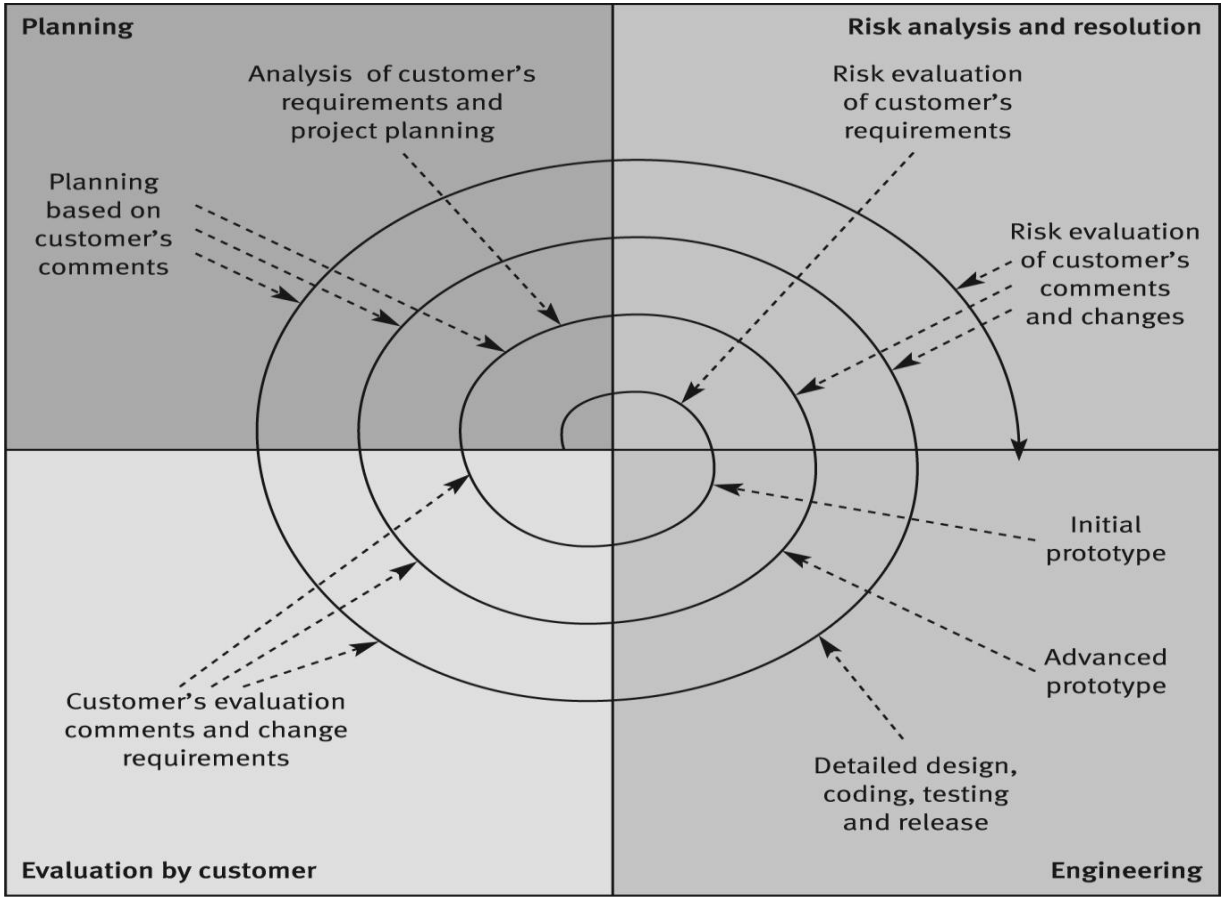
- **Waterfall:** traditional approach to build software
- **Spiral:** iterative approach to build software
- Other Agile methods
  - **SCRUM**
  - **eXtreme Programming** (XP)
  - **Rational Unified Process** (RUP) is an incremental and iterative approach
- DevOps

Which approach is this?



- Unidirectional, no way back
  finish this step before moving to the next

Which Approach is this?



Loops: Phases of the software

Radius: expenses or costs so far

Spiral Model

Image source: Unknown

UWE Bristol University of the West of England

What if customer want to see the real progress of his software?

Agile Software Development

- Agile SD is a way of thinking about project management
- Based on iterative and incremental development
- Requirements as well as solutions evolve together
- Collaboration between cross-functional, self-organising teams
- Teams kept small (5-9 people)

UWE Bristol University of the West of England

Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

http://agilemanifesto.org/

Agile Approach Overview

1st Iteration 2nd Iteration 3rd Iteration

UWE Bristol University of the West of England

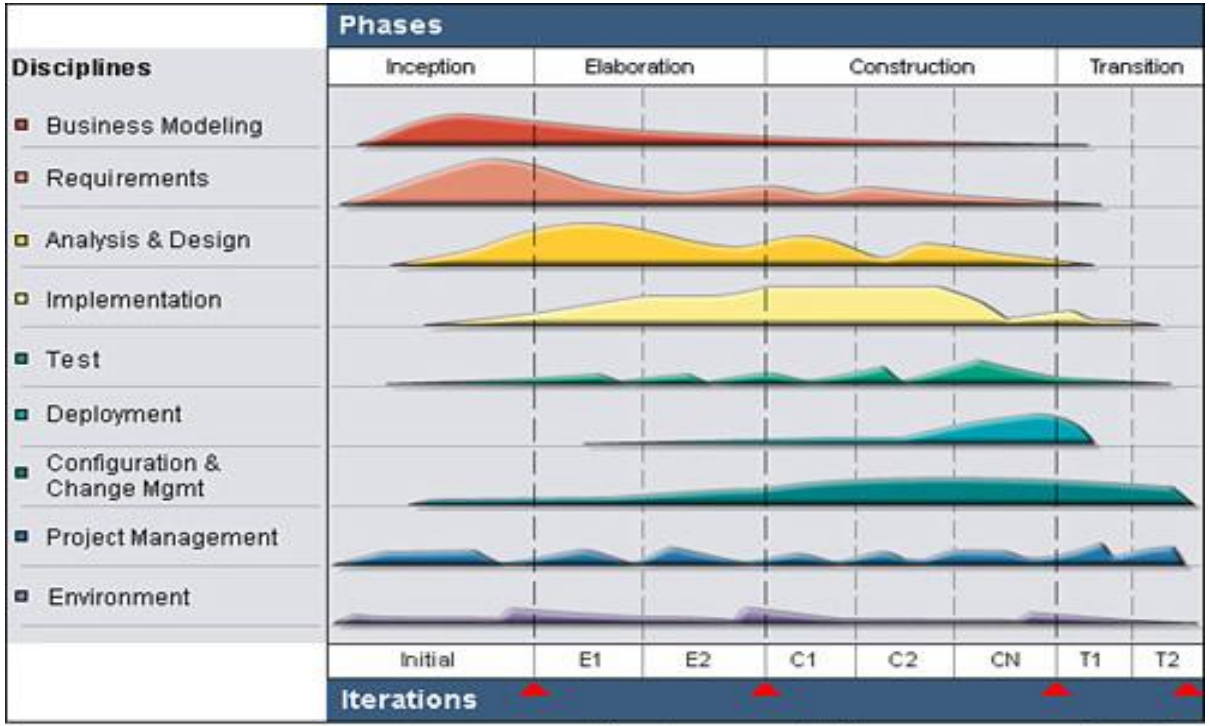
Agile Methods

RUP, Scrum, Xtreme Programming

Rational Unified Process (RUP)

- IBM Rational Unified Process (RUP) is an iterative and incremental software development method
  - It has phases, iterations in phases, milestones, disciplines, etc.
- Phases are made of iterations: Inception, Elaboration, Construction, Transition, Production
- Disciplines: *"a discipline is a collection of related activities that are related to a major area of concern". E.g., Requirements, Analysis and Design*
- Principles: *Architectural centric approach, component-based development*
- +ve
  - Good documentation, risk management support, reusability of components results in less total time duration
- -ve
  - Complex and team of professional is needed, more dependency on risk management

Rational Unified Process (RUP)



Source 1: https://www.redbooks.ibm.com/redbooks/pdfs/sg247362.pdf

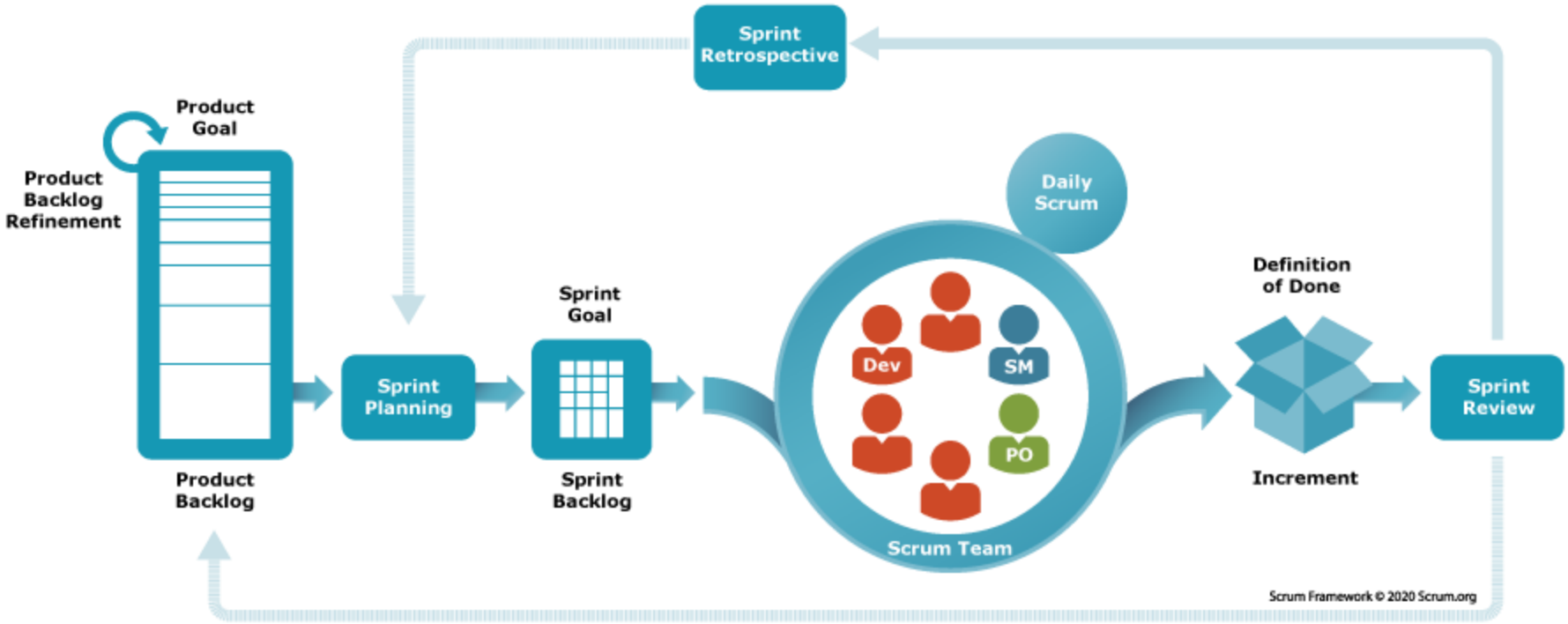
Source 2: https://www.informit.com/articles/article.aspx?p=1155863&seqNum=2

Scrum

- A typical, but detailed, Agile methodology
- Scrum is iterative philosophy. Iterate over changes, deployment, software development.
- Scrum is an iteration of Plan, Build, Test, and Review => Continuous iteration over these phases.
- Scrum pillars: Transparency, inspection and adaptation
- Scrum values: commitment, focus, openness, respect, and courage
- Manage a prioritized list of needs on a product backlog, collaborate through daily standup meetings, exhibit the product upon iteration completion, use retrospectives to correct the process
- See https://www.youtube.com/watch?v=XU0llRltyFM&t=0s for a very good <9 min. introduction to scrum

Read: Scrum guides: https://scrumguides.org/
Read: What is Scrum? https://www.scrum.org/resources/what-is-scrum

Scrum



Source: https://www.scrum.org/resources/what-scrum-module

Scrum Team

- No. 1: Product owner
- No. 2: The Scrum Master
- No. 3: Developers

- All have specific accountabilities
- Everyone should be able to work on everything

See an example: Khan, Z.; Dambruch, J.; Peters-Anders, J.; Sackl, A.; Strasser, A.; Fröhlich, P.; Templer, S.; Soomro, K. Developing Knowledge-Based Citizen Participation Platform to Support Smart City Decision Making: The Smarticipate Case Study. *Information* **2017**, *8*, 47. https://doi.org/10.3390/info8020047

Direct URL: https://www.mdpi.com/2078-2489/8/2/47/htm

Scrum Team-Product Owner

- Identifies product goal and maximising product value resulting from the work of the scrum team
- Creating and managing product backlog
- One person, not a committee
- may represent the needs of many stakeholders in the product backlog
- Product Owner must be convinced if someone wants to change the product backlog

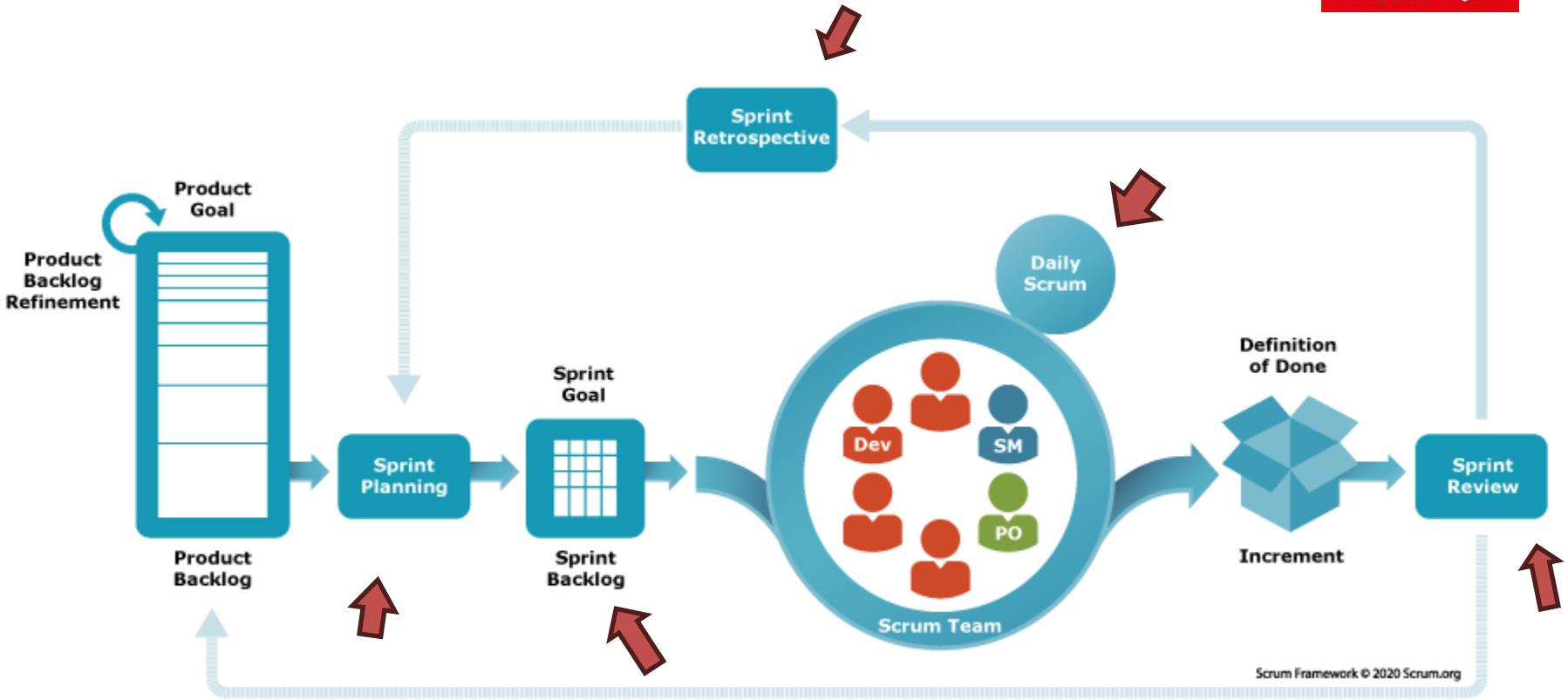
Scrum Team-Scrum Master

- Scrum master is responsible for the Scrum process
- Three main roles:
  1. Coach
  2. Fixer
  3. Gatekeeper
- Work in collaboration with Product Owner
- Two main tasks:
  1. Protecting team from outside disturbances
  2. Removing obstacles

Scrum Team-Developers

Team who will develop, test application

What else do we need in Scrum?



Source: https://www.scrum.org/resources/what-scrum-module

Scrum Events

- Create regularity and opportunities to inspect and adapt
- Minimise the need for meetings not defined in scrum
- The Sprints: heartbeat of scrum – to turn ideas into value
- Sprint planning: laying out the work to be performed… scrum team plans it
- Daily scrum: 15 minute event - inspection towards Sprint goal and adaptation of sprint backlog as necessary
- Sprint review: to inspect outcome of the sprint and determine future adaptations
- Sprint retrospective: plan ways to increase quality and effectiveness

Read: Scrum guides: https://scrumguides.org/

Scrum Artifact

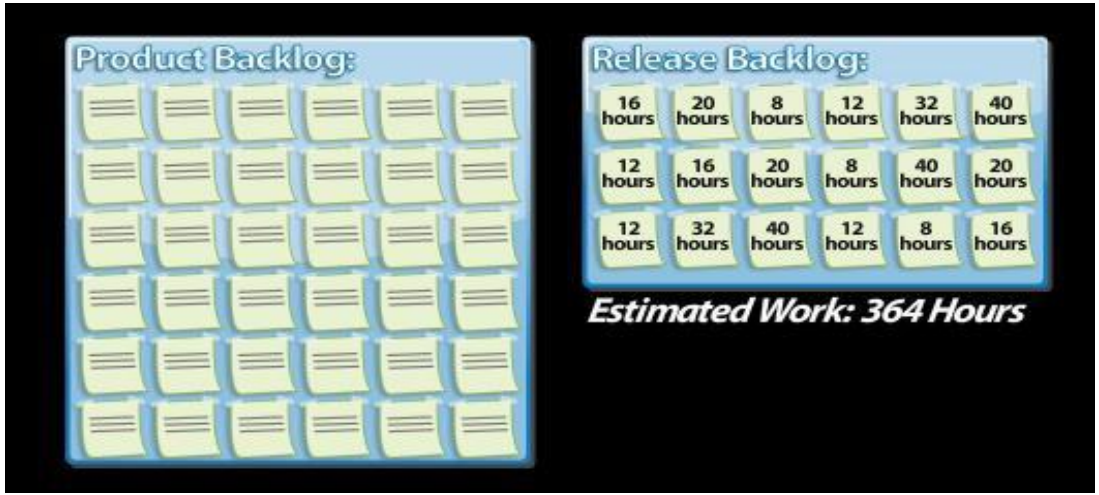
- Artifacts represent work or value
- Product backlog – emergent and ordered list of what is needed to improve the product; product goal – future state of the product and long-term objective of the scrum team
- *"A product is a vehicle to deliver value. It has a clear boundary, known stakeholders, well-defined users or customers. A product could be a service, a physical product, or something more abstract."*
- Sprint backlog – the set of product backlog items selected for the sprint (what), sprint goal (why) and actionable plan for delivering the increment (how)
- Increment – a concrete step towards achieving the product goal; Definition of Done – formal description of the state of the Increment when it meets the quality measures required for the product

Source: Scrum guides: https://scrumguides.org/

Product backlog

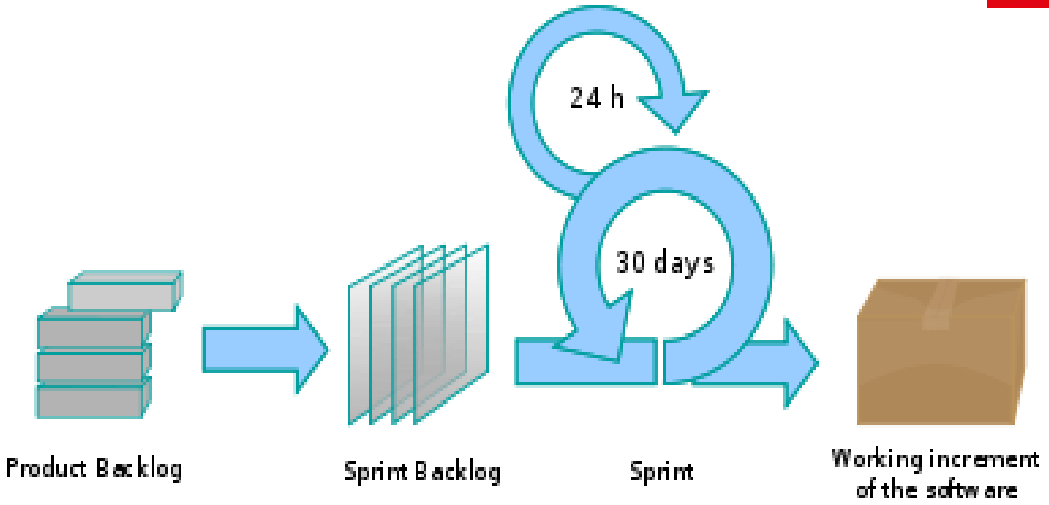
- Product backlog contains all user stories of features that the product needs/could have
- Product backlog is meant to grow over time
- **User stories examples:**
- **As <Who>,<What> and <Why?>**
- As manager, I should be able to add new cinema in a selected city, to increase business
- As admin, system should allow to add new listings, so that new shows and movies can be up for booking

Release planning



- Team and customer pick features from product backlog
- This creates a release backlog
- Release backlog features are assigned priority
- Release backlog features are assigned effort estimates
- Release is split in a number of sprints

Scrum sprint

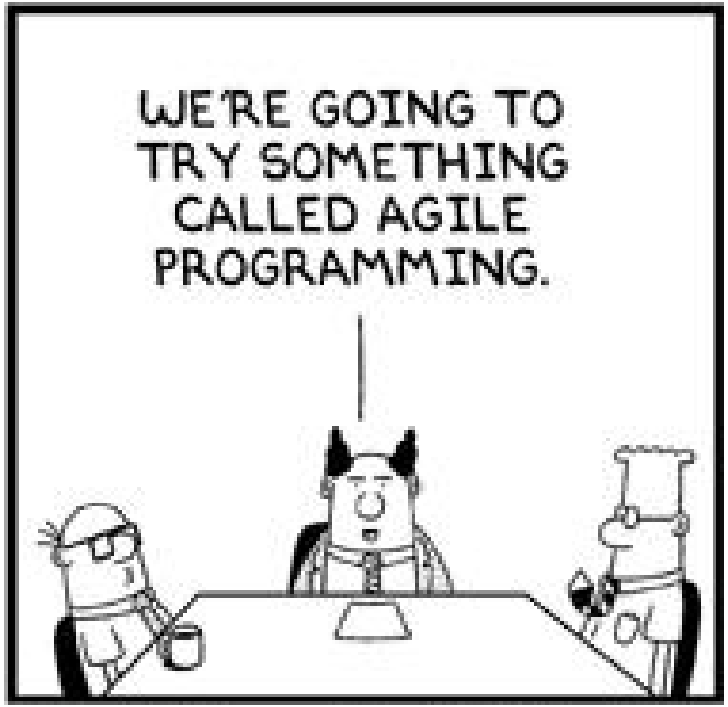


- Sprints last approximately 3 – 30 days
- Each release consists of 4-12 sprints
- Duration of sprint is proportional to release interval
- Sprint backlog lists features included per sprint

Daily Scrum

- Daily meeting

- Meeting should be held standing
  - Keep people on their toes
  - Keep meetings short and to the point
- Discusses what was done yesterday
- Discusses what will be done today
- Excellent tool to spot issues as they arise



Daily Scrum Questionnaire

Every team member has to answer three question:

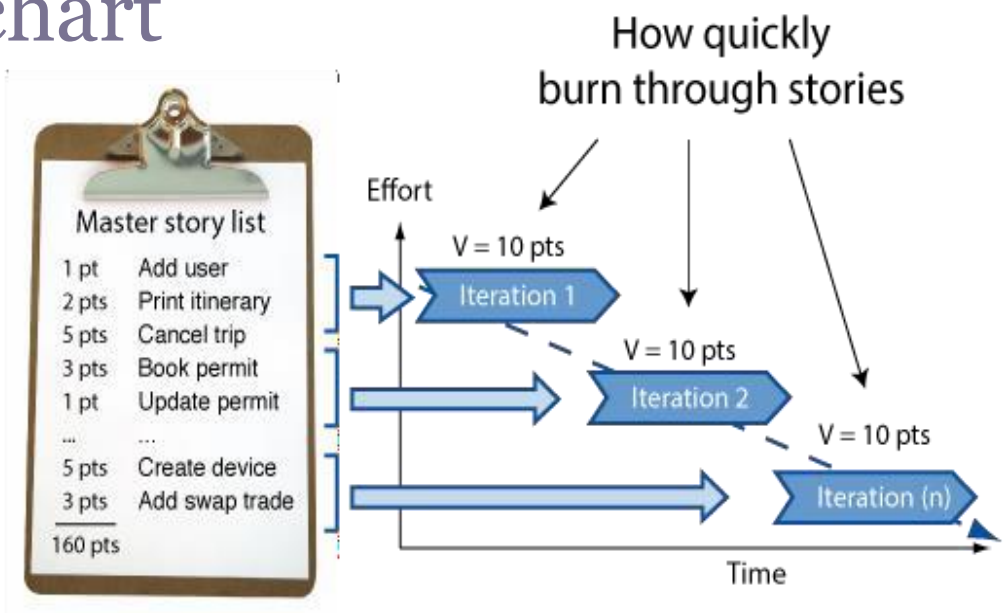
1. What have I done since last meeting?
2. What will I do until the next meeting?
3. What problems have I encountered?

During scrum meeting only team speaks.

Monitoring progress

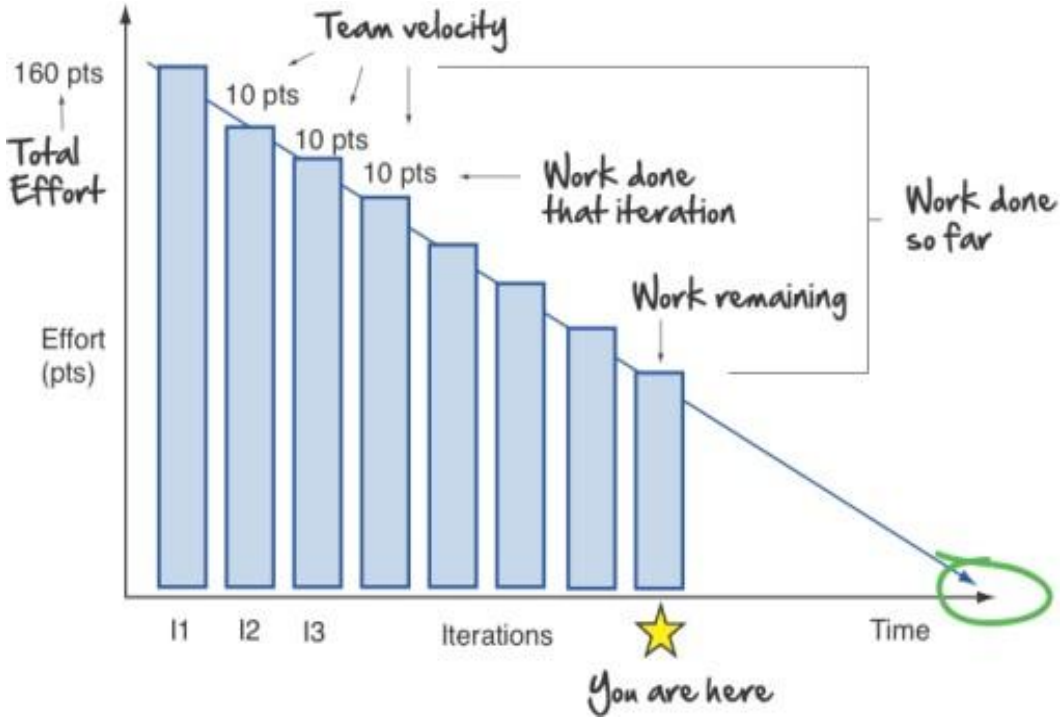
- Per the Agile manifesto, progress is measured in terms of functionality added
- Time to complete features in active sprints drawn in a burndown chart
- Burndown chart is updated daily
- Burndown velocity can be measured from chart
- Team estimates daily how much time is required per remaining feature

Burndown chart



- The burndown is a chart that shows how quickly you and your team are burning through your customer's user stories.
- It shows the total effort against the amount of work we deliver each iteration

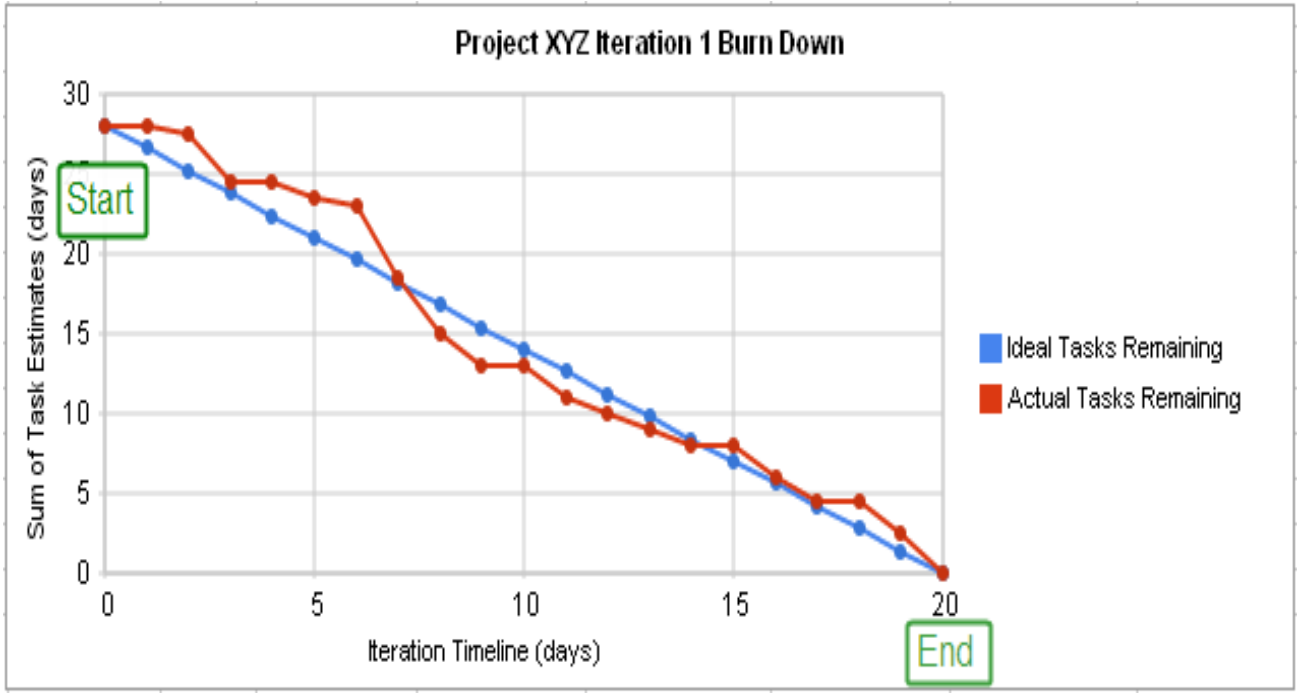
Burndown chart cont.



The speed at which we turn user stories into working software is called the **team velocity**.

Planning:
iterations = total effort / estimated team velocity

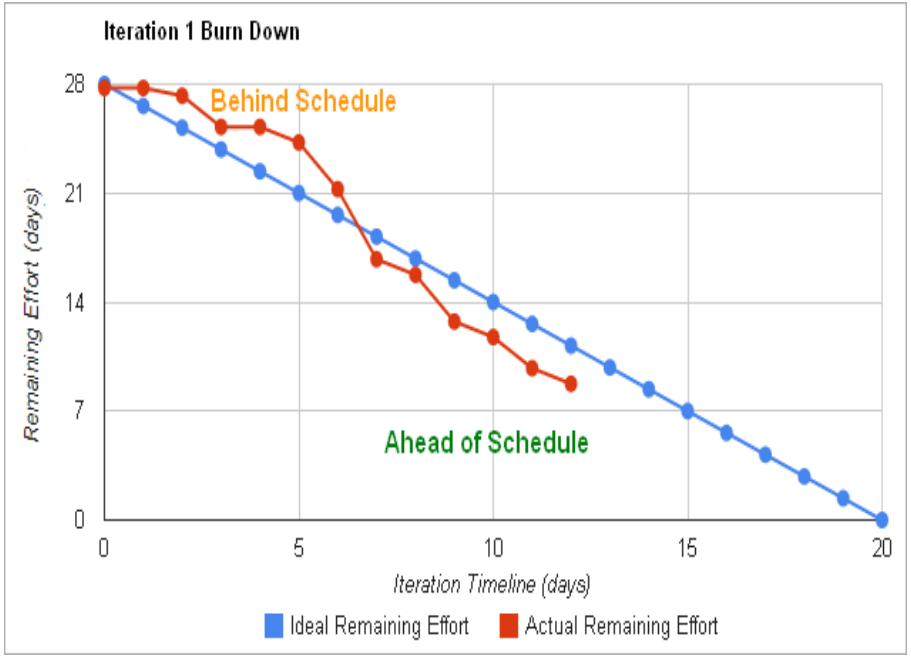
Burndown chart cont.

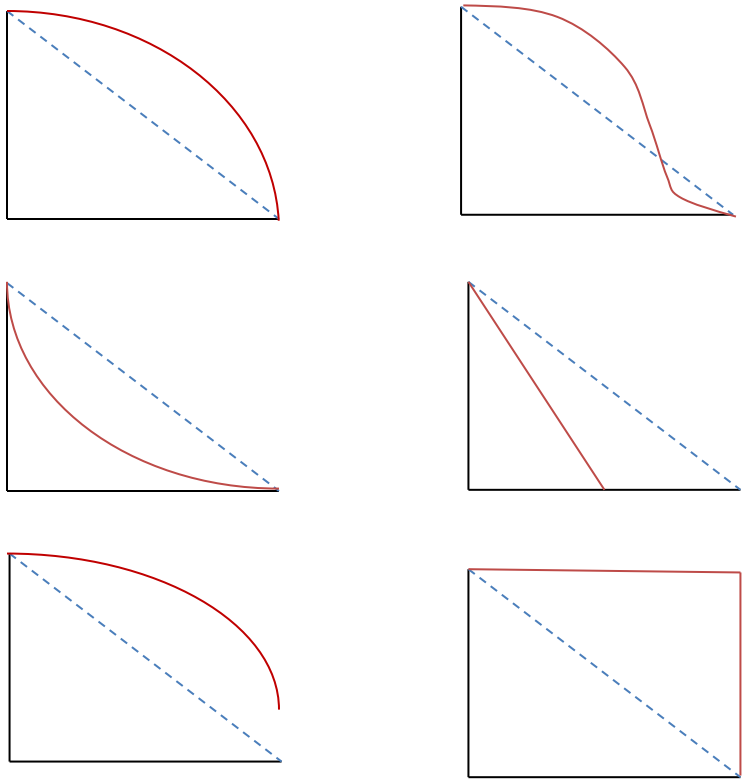


http://www.methodsandtools.com/archive/scrumburndown.php

Image&Texts source:inpointform.net

Burndown chart cont.





Images&Texts source: http://www.methodsandtools.com/archive/scrumburndown.php

Image&Texts source: inpointform.net

Scrum for larger projects!

UWE Bristol University of the West of England

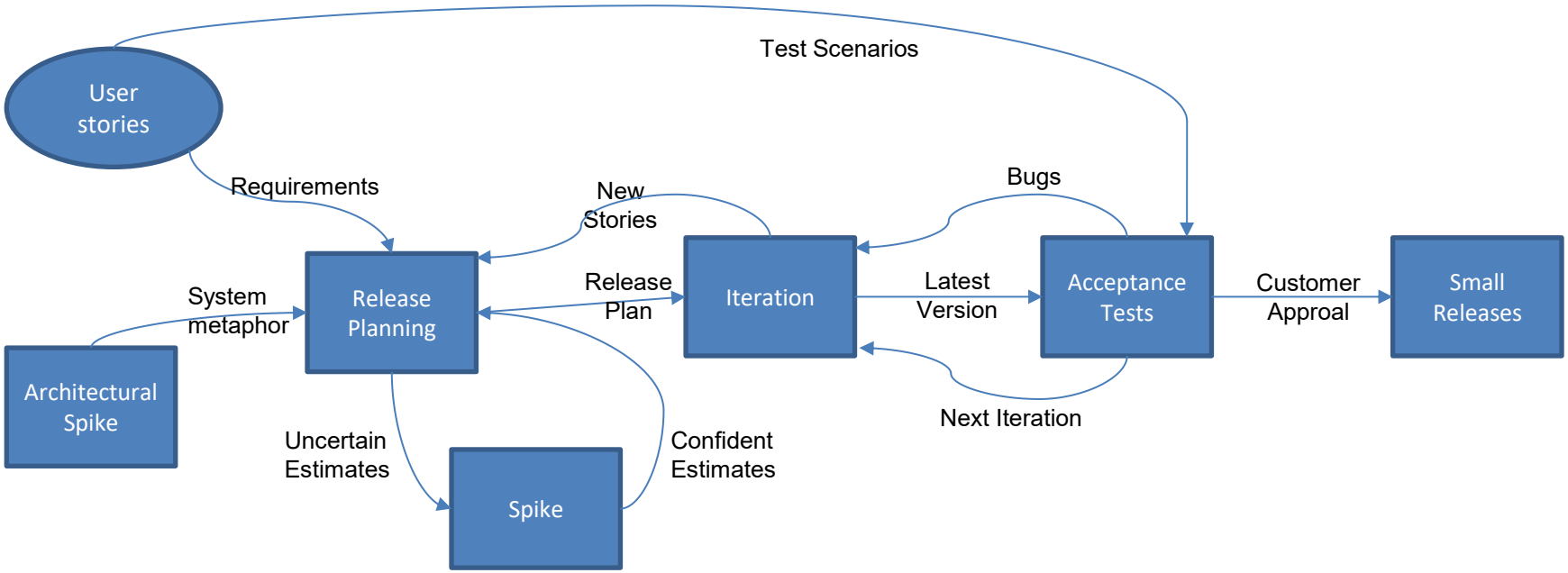
- We agree that Scrum is ideal for small sized project
- What about when we need a very large team working on a single project, or on closely related projects in a large programme?
- Possibly using Scrum of Scrums technique
- Each team meets every day and holds their daily Scrum as usual. One or two representatives from each Scrum team attend a higher level Scrum to coordinate across teams. And on very large teams, one or two representatives from the higher level Scrum attends an even higher level Scrum, and so on

Extreme programming (XP)

- Perhaps the best-known and most widely used agile method.
- Extreme Programming (XP) takes an 'extreme' approach to iterative development.
  - New versions may be built several times per day;
  - Increments are delivered to customers every 2 weeks;
  - All tests must be run for every build and the build is only accepted if tests run successfully
- Software should be tested frequently during development
  - Extreme programming advocates testing code literally every few minutes, after every minor change
- Extreme programming works best for relatively small projects with a small number of good programmers

Source and Read: https://www.digite.com/agile/extreme-programming-xp/

Extreme programming (XP)



Source and Read: https://www.digite.com/agile/extreme-programming-xp/

XP practices

- The XP practices we will emphasize are:
  - **Pair Programming**
    - Teams of two people
  - **Test Driven Development**
    - Writing lots of tests, and writing them early
  - **Continuous Integration**
    - Putting code together as you write it, not at the last minute
  - **Coding Standards**
    - Learn and follow well-established conventions (e.g., camel notation)
  - **Collective Code Ownership**
    - You are responsible for your partner's code
  - **Simple Design**
    - Stay away from confusion

Pair programming

- **Pair programming** is an agile software
- development technique in which two programmers
- work together at one workstation.

- One, the **driver**,
- writes code while the other, the **observer** or **navigator**,
- reviews each line of code as it is typed in.

- The two programmers switch roles frequently.
- While reviewing, the observer also considers
- the strategic direction of the work, coming up
- with ideas for improvements and likely future problems to address.
- --Laurie Williams
  North Carolina State University

Texts&Image source: wikipedia.org

- Computer Science

Differences Between Scrum and XP

- Are there any differences between Scrum and XP?

Differences Between Scrum and XP

UWE Bristol University of the West of England

- Scrum teams typically work in iterations (called sprints) that are from two weeks to one month long. XP teams typically work in iterations that are one or two weeks long.

- Scrum teams do not allow changes into their sprints. Once the sprint planning meeting is completed and a commitment made to delivering a set of product backlog items, that set of items remains unchanged through the end of the sprint.

- XP teams are much more amenable to change within their iterations. As long as the team hasn't started work on a particular feature, a new feature of equivalent size can be swapped into the XP team's iteration in exchange for the unstarted feature.

Do you see any drawbacks of agile methods?

DevOps

Traditional Software Development Environment

- Traditional software development comprises of Development Team and Operations Team
- Development Team is responsible for plan, design ,and build a software
- Operations team deploy and manage the software

- Challenge of working in traditional environment?

Operations and Development teams have little-to-no direct interaction with each other which introduces several problems. For example, Developers team expect operations team to release new applications even though the code wasn't thoroughly tested. Similarly, operations managers, who consider frequent code updates risky, ignore change requests from the development side resulting in holding up new releases.

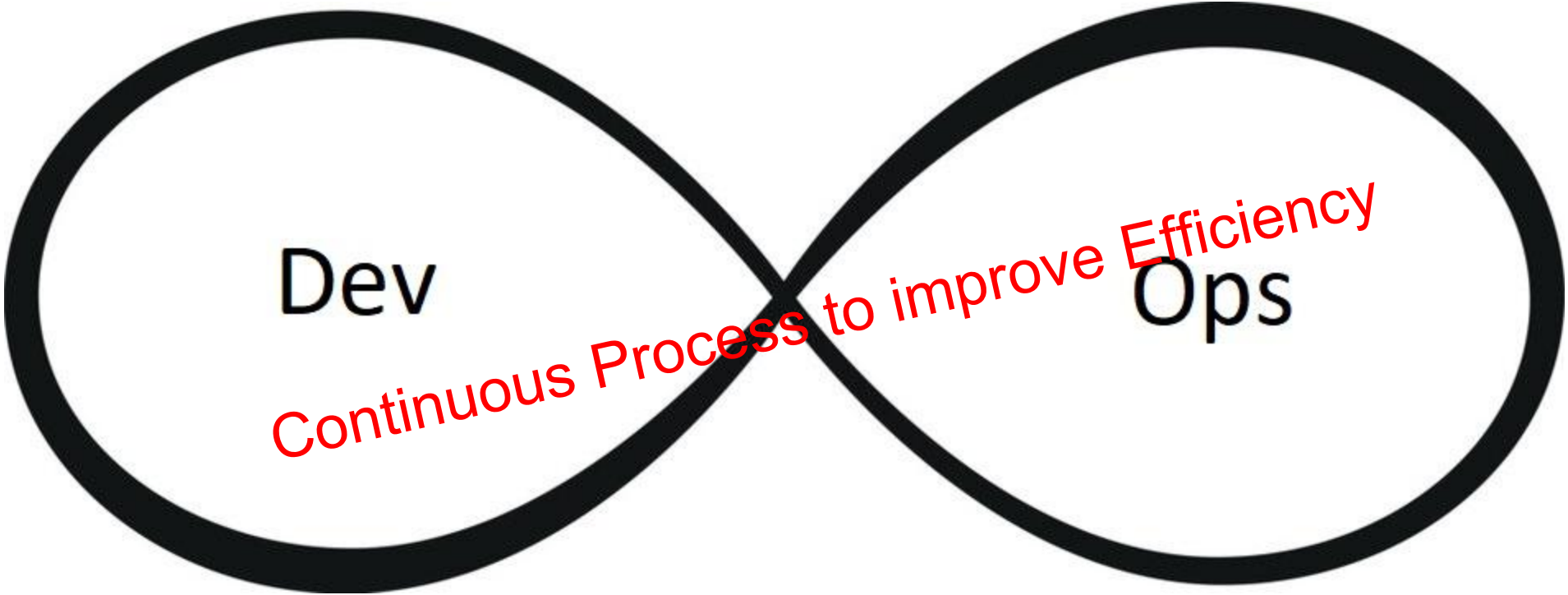
How to Address Traditional Software Development Environment Challenges?

Work In Collaboration

Development Team

Operations Team

DevOps Overview



What is DevOps Methodology?

UWE Bristol University of the West of England

- ***DevOps is a combination of software development (dev) and operations (ops). It is defined as a software engineering methodology which aims to integrate the work of development teams and operations teams by facilitating a culture of collaboration and shared responsibility.***
- DevOps team reduces a gap between development and operations personnel through a combination of processes, cultural enhancements, and supporting technologies
- Stems from an Agile approach to software development
- DevOps focuses on incremental development and rapid delivery of software.
- DevOps process expands on the cross-functional approach of building and shipping applications in a faster and more iterative manner.
- DevOps encourages collaborative environment at all stages of the development cycle.

Source: https://about.gitlab.com/topics/devops/

Key Principles of DevOps

i. **Automation of the software development lifecycle:** automating testing, builds, releases, the provisioning of development environments, and other manual tasks that can slow down or introduce human error into the software delivery process.

ii. **Collaboration and communication:** breaking down silos and making the development life cycle faster, highly automated, and seamlessly collaborative – whether it's cross-functional or within one team.

iii. **Continuous improvement and minimization of waste:** High performing DevOps teams regularly look for areas of improvement such as by automating repetitive tasks, reduce release time.

iv. **Hyperfocus on user needs with short feedback loops:** First three principles help DevOps teams to focus on user needs

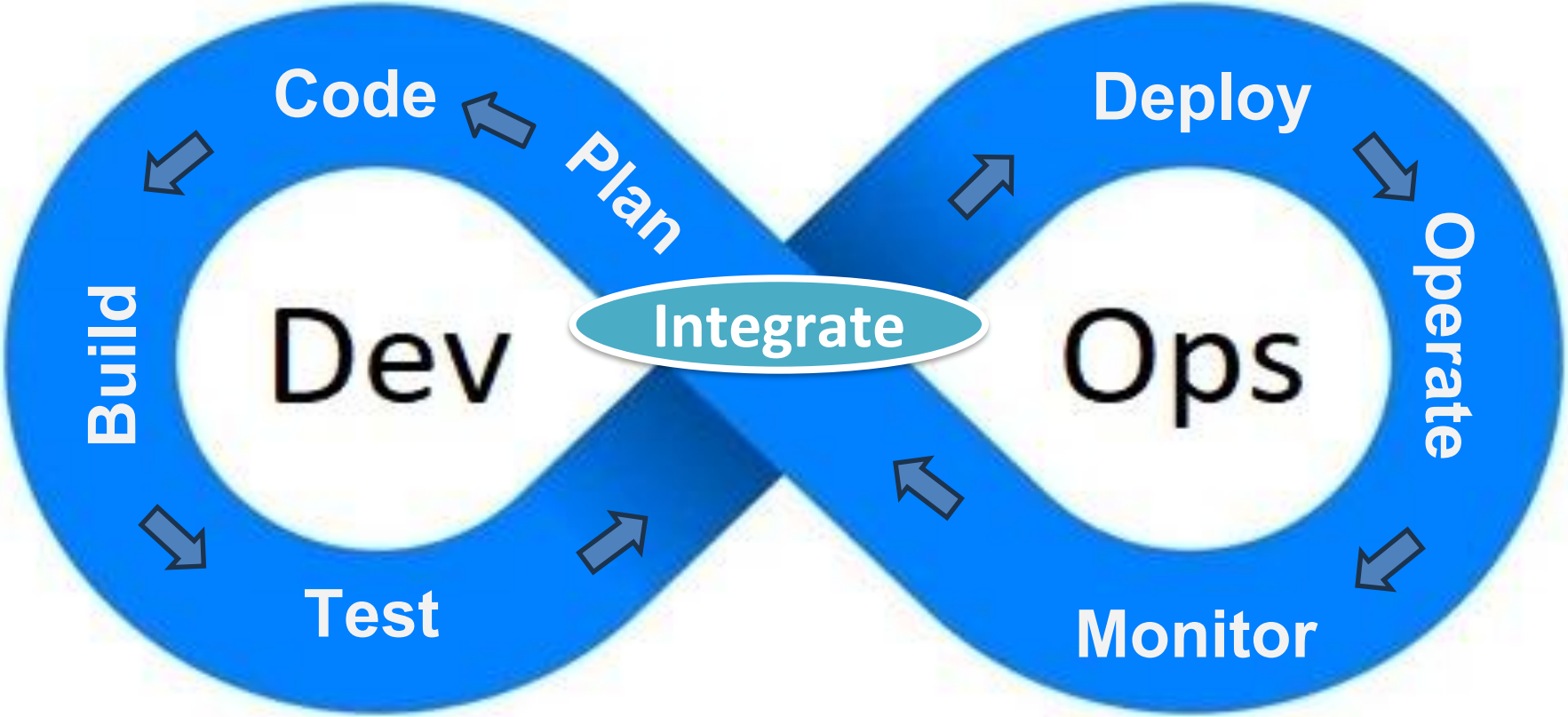
Source: https://about.gitlab.com/topics/devops/

DevOps Team

- A DevOps team includes developers and IT operations working collaboratively throughout the product lifecycle, to increase the speed and quality of software deployment.

- DevOps teams use tools to automate and accelerate processes, which helps to increase reliability.

DevOps Lifecycle



DevOps Lifecycle

Plan: This phase includes planning by keeping in view customers' objectives
Code: This is development phase
Build: Code is made executable
Test: Testing of code for bugs and errors.
Deploy: Ready for deployment
Operate: deploys code to working environment
Monitor: Product continuously being monitor.
Integration Phase: This is Core of DevOps. Feedback received at Monitoring phase is sent back to Planning phase.

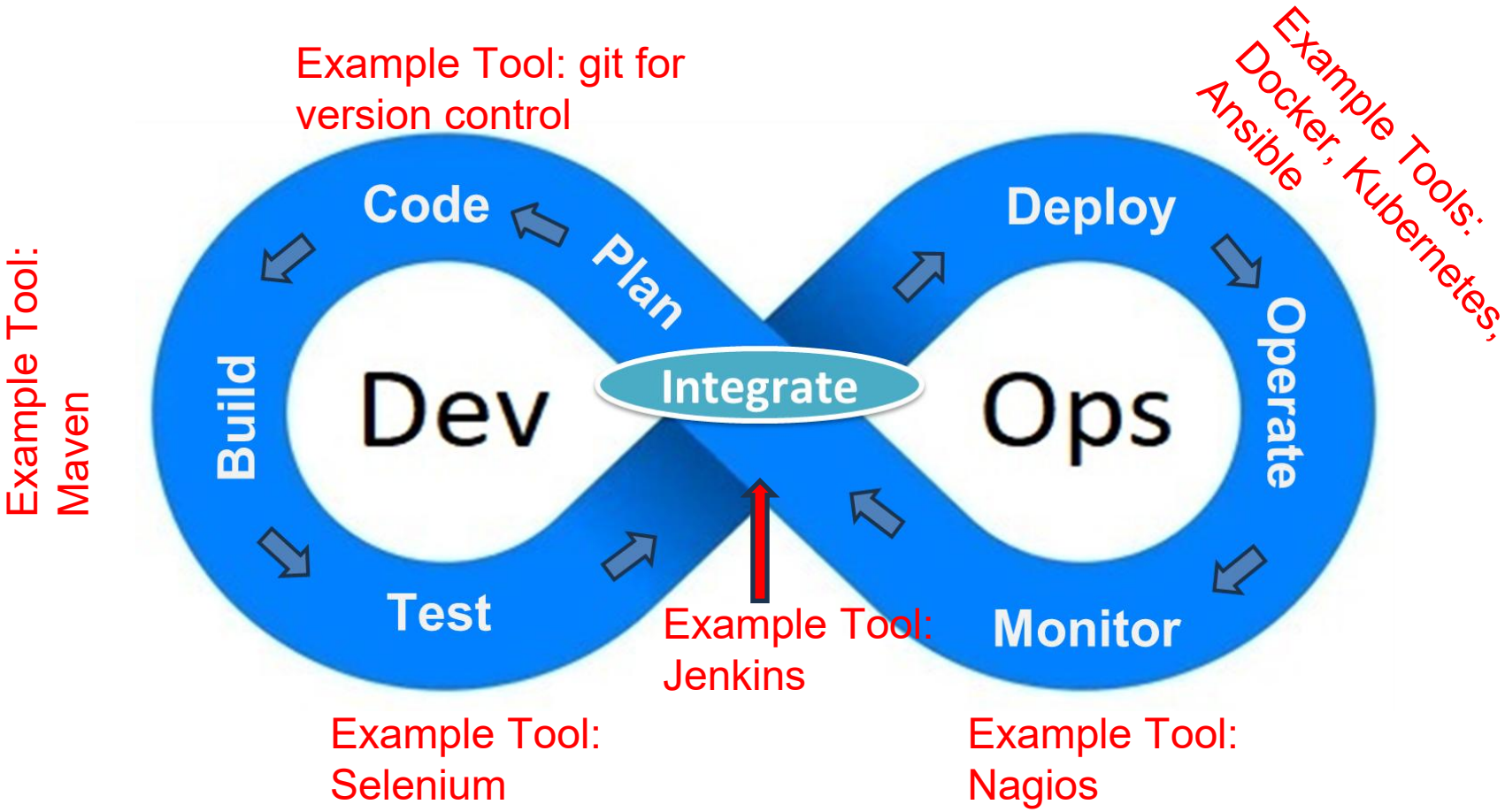
DevOps Toolchain

UWE Bristol University of the West of England

A DevOps toolchain is a collection of tools, often from a variety of vendors, that operate as an integrated unit to design, build, test, manage, measure, and operate software and systems.

A DevOps toolchain helps teams tackle important DevOps fundamentals including continuous integration, continuous delivery, automation, and collaboration.

Example Tools used in DevOps Phases



Automation Challenges in DevOps

UWE Bristol University of the West of England

All-in-one DevOps toolchain- *provides a complete solution that may not integrate with other third-party tools. But one tool can't evolve to fast-changing markets. Another challenge is Integration of legacy tools into DevOps toolchain. Most established teams already have a set of tools they use and prefer, which may not integrate with a complete solution.*

Customisable DevOps toolchain- *DevOps toolchain that can be customized for a team's needs with different tools. Integration is essential for these types of toolchains. If the different tools don't integrate, team members spend unnecessary time switching between screens, logging in to multiple places, and it may be challenging to share information between tools*.

DevOps Platform

GitLab’s DevOps platform brings tools together in a single application.

DevOps platform allow visibility and control over all stages of DevOps lifecycle.

DevOps platform empowers teams to collaboratively plan, build, secure, and deploy software

Best Practices for Implementing DevOps

UWE Bristol University of the West of England

Agile project management- focus on delivering work in smaller increments

continuous integration and continuous delivery (CI/CD)- Continuous integration (CI) allows multiple developers to contribute to a single shared repository. When code changes are merged, automated tests are run to ensure correctness before integration. Continuous deployment (CD) allows teams to release features frequently into production in an automated fashion.

Build with the right tools- A DevOps toolchain requires the right tools for each phase of the DevOps lifecycle

Implement automation- Continuous integration and delivery allows developers to merge code regularly into the main repository. Instead of manually checking code, CI/CD automates this process, from batching in a specified window to frequent commits. In addition to CI/CD, automated testing is essential to successful DevOps practices. Automated tests might include end-to-end testing, unit tests, integration tests, and performance tests.

Best Practices for Implementing DevOps

Monitor the DevOps pipeline and applications- *Automation improves the speed of development tremendously, but if there is a failure in an automated process and nobody knows about it, working manually is better.*

Gather continuous feedback- *Continuous feedback ensures team members have all the information needed to do their jobs on a timely basis.*

Organisation culture change- *DevOps requires collaboration, transparency, trust, and empathy.*

Benefits of Adopting DevOps

Speed- Continuous delivery allows teams to build, test, and deliver software with automated tools.

Improved Collaboration- Shared responsibilities between development and operations teams

Rapid Deployment- quickly releasing new features and repairing bugs.

Quality and Reliability- CI/CD ensure changes are functional and safe, which improves the quality of a software product.

Challenges of Adopting DevOps

UWE Bristol University of the West of England

For teams working in Siloed ways, it is challenging to embrace DevOps practices

Some teams may mistakenly believe new tools are sufficient to adopt DevOps.

Automation Challenges

Summary

- Agile software development is a software development/project management approach
- Agile projects incrementally create an evolving product
- Agile uses small cross-functional teams with heavy commitment of the customer
- Scrum/XP Agile methodologies
- DevOps
- Next: Object-oriented modelling

More on Software Development Models https://www.geeksforgeeks.org/software-engineering-classical-waterfall-model/?ref=lbp

More on Agile: https://www.tutorialspoint.com/agile/index.htm

More on DevOps: https://about.gitlab.com/topics/devops/ (8 mins read)